

Vision Based Neuro-Fuzzy Controller for a Two Axes Gimbal System with Small UAV

Ashraf Qadir · William Semke ·
Jeremiah Neubert

Received: 30 July 2012 / Accepted: 9 July 2013
© Springer Science+Business Media Dordrecht 2013

Abstract This paper presents the development of a vision-based neuro-fuzzy controller for a two axes gimbal system mounted on a small Unmanned Aerial Vehicle (UAV). The controller uses vision-based object detection as input and generates pan and tilt motion and velocity commands for the gimbal in order to keep the interest object at the center of the image frame. A radial basis function based neuro-fuzzy system and a learning algorithm is developed for the controller to address the dynamic and non-linear characteristics of the gimbal movement. The controller uses two separate, but identical radial basis function networks, one for pan and one for tilt motion of the gimbal. Each system is initialized with a fixed number of neurons that act as rules basis for the fuzzy inference system. The membership functions and rule strengths are then adjusted with the feedback from the visual tracking system. The controller is trained off-line until a desired error

level is achieved. Training is then continued on-line to allow the system to accommodate air speed changes. The algorithm learns from the error computed from the detected position of the object in image frame and generates position and velocity commands for the gimbal movement. Several tests including lab tests and actual flight tests of the UAV have been carried out to demonstrate the effectiveness of the controller. Test results show that the controller is able to converge effectively and generate accurate position and velocity commands to keep the object at the center of the image frame.

Keywords Unmanned Aerial Vehicle (UAV) · Zero Mean Normalized Cross Correlation (ZMNCC) · Neuro-Fuzzy system · Radial Basis Function Network (RBFN)

1 Introduction

1.1 Motivation

Onboard visual tracking with small UAV (Unmanned Aerial Vehicle) has great potential in areas such as search and rescue, traffic control and military applications for tracking non-cooperative objects, i.e. objects that do not provide their GPS location. In visual tracking systems, the object of interest is detected in one image frame and

A. Qadir (✉) · W. Semke · J. Neubert
Mechanical Engineering Department,
University of North Dakota, Grand Forks,
ND, USA
e-mail: ashraf.qadir@my.und.edu

W. Semke
e-mail: william.semke@engr.und.edu

J. Neubert
e-mail: jeremiah.neubert@engr.und.edu

then tracked in subsequent frames. To have a successful tracking system, the object being tracked needs to be kept inside the camera view. When the camera is mounted on a gimbal, the visual information derived from the processing of images is used to control the motion of the gimbal to track and keep the object at the center of the image frame. However, the relative velocity of the object is nonlinear as the speed and direction of the UAV and object varies. A control system is needed for the variable speed and motion of the camera in order to keep the object at the center of the image frame.

1.2 State of the Art in Steering Laws for Visual Tracking Systems

The on-board visual tracking system requires the object to be within the camera field of view for successful tracking. The tracking system detects the object in the image frame and uses the information to steer the gimbal in order to keep the object at the center of the image frame. The steering law requires the state (position, velocity) estimation of the target with respect to the gimbal rotation center at each instance. A considerable amount of research work has been done on object location estimation using the camera on-board the UAV. Some early systems [1, 2] used registration of the information collected by the on-board camera with pre-existing geo-reference information for geo-location estimation of the target object. Stolle and Rysdyk [3] present a flight path design for UAV with pan-tilt camera for target observation. A control algorithm for the camera motion was developed in order for the UAV mounted camera to point at a known target location. The pointing algorithm depends on the aircraft attitude information based on the GPS data. The simulation results were presented and localization accuracy was not discussed. Dobrokhodov et al. [4] developed a vision based target tracking system with small UAV. The system uses a terrain model for the range estimation and a non-linear filter was used to estimate the position and velocity of the moving object on the ground. Then a control algorithm steers the gimbal and the UAV in order to keep the object at the camera line of sight. A vision based target geo-location estimation method

has been developed by Barber et al. [5] using GPS, IMU and vision sensor that does not require pre-existing geo-reference imagery or terrain model. Assuming a flat earth model, the object state relative to the camera center is estimated and then transformed to state in inertial coordinate using the GPS, IMU, pressure sensors and a set of coordinate transformations. Uncertainties in estimation relative to different flight conditions and sensor measurements are also explained in details in [5, 6]. In the previous work [7] by the authors, an on-board visual tracking system was developed that is capable of tracking real time and reduces the dependency on communication with ground. The camera was mounted on a three axes gimbal. The position of the object in the image frame was detected and the distance from the center of the image frame was computed. The position command for the gimbal was generated and the gimbal was moved accordingly. However, a fixed velocity was used to move the gimbal and the system was unable to generate an accurate motion of the gimbal in order to track different speed objects and keep them at the center of the image frame.

1.3 State of the Art in Neuro-Fuzzy Control

One of the earliest architectures for neuro-fuzzy system is the adaptive-network-based fuzzy inference system (ANFIS) [8]. This system implements a first-order T-S [9] fuzzy system as the inference part and a hybrid learning rule is used to train all the parameters in ANFIS. In this approach, the number of membership functions for each input variable has to be assigned in advance. Leng et al. [10] has proposed neuro-fuzzy system for on-line extraction of fuzzy rules from training data. The parameters are then updated using linear least square method. Radial basis function neural network (RBFNN) is receiving increasing attention in neuro-fuzzy control of nonlinear systems due to their simple structure, good local approximation capabilities and functional equivalence with fuzzy systems [11–13]. Jang and Sun [12] shows functional equivalence between Radial Basis Function Networks (RBFN) and fuzzy systems from the similarity between local receptive fields of the RBFN and the membership functions of the fuzzy

system. Cho and Wang [13] proposed a radial basis function (RBF) based adaptive fuzzy inference system. In this model an RBF neuron is added to the network by the hierarchically structured self-organizing learning method. Then the parameters of the system as mean and variance vectors as well as the weights are updated using gradient descent algorithm.

A substantial amount of research work has been carried out in adaptive, intelligent control of nonlinear and time varying systems using neural network and fuzzy logic. Major approaches include direct adaptive control scheme [14–24] and indirect adaptive control schemes. In indirect control scheme, the system model is estimated online and then control parameters are generated based on the current estimated model whereas the control parameters are estimated in the direct adaptive control scheme [25–30] without knowledge of the dynamics of the system. Recently, high order neural network (HONN) based fuzzy system is proposed for dynamic systems approximation [31–33] where the systems are approximated by a fuzzy system and the HONN is used to identify the fuzzy rules. Theodoridis and Boutalis [33] proposed a HONN based neuro-fuzzy system for nonlinear dynamic system control where both the parameter and dynamic uncertainties are considered. A neural network structure with an activation feedback and an output feedback to imitate the behavior of the classical PID controller is proposed in [34]. The network weights are updated online from the modeling and external disturbance errors while the stability of the learning rate is obtained using Lyapunov stability theory.

Last few years have seen the application of neuro-fuzzy controller in a wide range of engineering control problems such as robot trajectory tracking and motion control [35–41], unmanned aerial and underwater vehicles and manipulator control [42, 43], vibration control [44] etc. A sensor based neuro-fuzzy controller for robot manipulator control has been described in [35] where a neuro-fuzzy controller generates the commands for servo-systems of the robot to move among dynamic obstacles and reaches its goal position. The controller shows its adaptability to change in manipulator dynamics. In [41] an adaptive controller has been developed for generating linear

and angular velocity commands for a unicycle-like robot. The system uses a two stage kinematic and dynamic controllers where the dynamic model parameters are updated online to account for the disturbances and uncertainty in robot dynamics. A neuro-fuzzy controller has been proposed in [43] for an underwater vehicle manipulator system (UVMS). The system uses a fuzzy PD controller where a set of fuzzy rules have been used for the gain tuning of the PD controller. A neural network for each link is also used to approximate the dynamics of each link and act as a compensator element to the controller output. However, only simulation results were presented in the paper. Collani et al. [45] developed a neuro-fuzzy system as fine motion controller based on vision and force sensors. A B-spline neuro-fuzzy model have been used as robot displacement controller where visual features are extracted from captured images and fed to the neuro-fuzzy controller. The controller learns on-line from measured forces and compensates motion accordingly. Lin et al. [46] proposed another fuzzy neural network based motion control system for linear ultrasonic motor. An adaptive interval type-2 fuzzy neural network (AIT2FNN) control system has been used where the parameters of the network are trained using the Lyapunov stability theorem. Wu and Gao [47] developed a vision based neuro-fuzzy controller to control the weld penetration in gas tungsten arc welding process of thin sheets. Weld pool images are captured and pool surface geometrical parameters are extracted. Weld joint penetration is then computed and a neuro-fuzzy controller is used to control the weld penetration process.

1.4 Our Approach

In the context of these recent developments, we propose a novel neuro-fuzzy control algorithm for the two axes gimbal movement using only a vision sensor onboard the UAV, and in the absence of a mathematical model of the system. The inputs to the controller consist of the detected object position and change in position errors in two successive image frames; and the controller generates position and velocity command for the gimbal motion controller that in turns moves the camera. A RBF based neuro-fuzzy system has

been developed for the controller. The radial basis function is developed in a form that acts as an inference engine for our neuro-fuzzy controller. A learning algorithm is then developed to train the membership functions and connection weights. The learning algorithm is similar to the parameters learning methods described by Cho and Wang [13] and by Wu and Gao [47].

1.5 Organization of the Paper

The paper is organized as follows; the basic visual tracking system is described briefly in Section 2. A mathematical model of the system is presented to better explain the problem formulation. Section 3 presents the detailed neuro-fuzzy controller architecture that has been developed, the implementation of the controller is described in Section 4, testing of the controller performance and results are shown in Section 5; finally, conclusion with future works is discussed in Section 6.

2 Overview of the Tracking System

2.1 Onboard Visual Tracking System

The onboard tracking system architecture is explained in [7]. The system architecture can be described in brief as:

1. Acquire image
2. Detect the object
3. Estimate gimbal motion needed to move the object to the image center
4. Actuate the camera

A schematic of the tracking algorithm used is shown in the Fig. 1.

The tracking system detects the object using zero mean normalized cross correlation [48]

where the template is slid over the image frame and correlation coefficient is computed to detect the match. The equation for computing the correlation coefficient between the image $f(x, y)$ and the template t positioned at u, v .

$$C = \frac{\sum [f(x, y) - \bar{f}_{u,v}] [t(x - u, y - v) - \bar{t}]}{\left\{ \sum [f(x, y) - \bar{f}_{u,v}]^2 [t(x - u, y - v) - \bar{t}]^2 \right\}^{1/2}} \quad (1)$$

where C is the zero mean normalized cross correlation coefficient, $\bar{t}$ is the mean intensity value of the template and $\bar{f}_{u,v}$ is the mean intensity value of the image $f(x, y)$ in the region under the template. A Kalman filter [49] is used to limit the area of the image searched and reduces the computation required by the tracking system. The position of the object of interest is predicted using a constant velocity motion model and the state is then updated using the measurement from the object detection algorithm. The distance between the detected position and the center of the image frame is computed and position and velocity commands are generated for the motor controller. This information is then sent to the motor controller to actuate the gimbal.

2.2 Mathematical Model of the System

The model shown in Fig. 2, an extension of the model presented in [5], will be used to better describe our work. The inertial frame is the fixed earth frame used by the GPS system. The location of the GPS antenna is at the center of the body fixed frame of reference, abbreviated as b in this work. The frame denoted with a g is the base of the gimbal and c is the camera frame. The camera frame origin is at the optical center of the camera

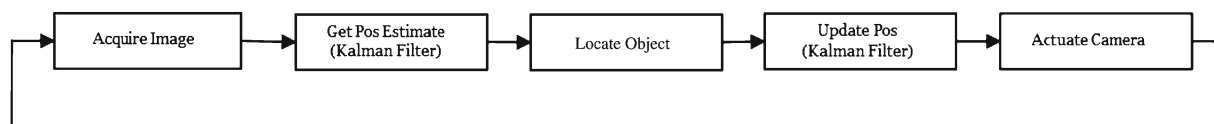


Fig. 1 Basic overview of the tracking system. When the new frame is acquired the position of the object is predicted with the motion model in Kalman filter. After locating the

object with template matching the position is updated with Kalman filter

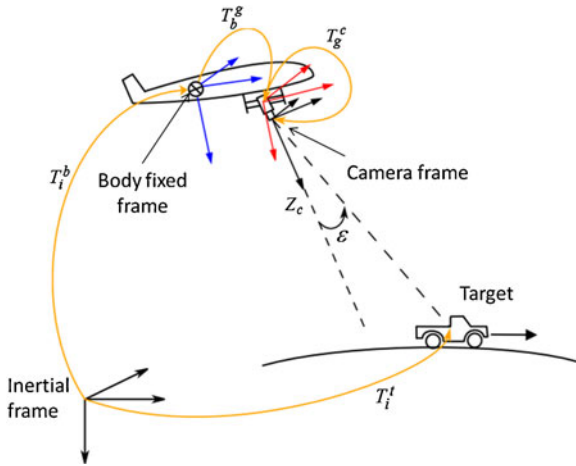


Fig. 2 Coordinate systems and transformations associated with the model. The camera frame is relative to the gimbal and ε denotes the angle between the target and pointing direction of the camera

with z-axis coincident with the optical axis of the camera. Another coordinate frame in the model is fixed to the target, denoted t in the Fig. 1. The objective is to manipulate the pan and tilt of the gimbal to minimize ε , the angle between Z_c and the line defined by the camera frame origin, o_c , and the target frame origin, o_t . Specifically, the objective of the neuro-fuzzy system is to produce the velocity and position commands needed to drive ε to zero. To obtain the pan-tilt angles needed to minimize ε , full states (position, velocity and acceleration) of the aircraft and the target are needed.

The objective of the mathematical model is to find the target pose relative to the camera frame. In order to obtain the target pose in the camera frame, the position of the camera center at each instance is required. The state of the camera relative to the target can be determined by making observations of the camera's position relative to the target. The camera position and orientation relative to the inertial frame are obtained from the sensor inputs and can be described using a homogeneous transformation matrix as

$$T_g^i = \begin{bmatrix} R_c^i & \vec{d}_c^i \\ \vec{0} & 1 \end{bmatrix} = T_b^i T_g^b \quad (2)$$

where the transformation from inertial to camera frame is denoted as T_c^i which consists of a

set of transformations from inertial to body fixed frame T_b^i , from body fixed to gimbal frame T_g^b , and from gimbal to the camera center frame T_c^g . The homogeneous transformation consists of a rotation matrix $R_c^i \in SO(3)$ that represents the orientation of the camera frame relative to the inertial frame. The column vector $\vec{d}_c^i \in \mathbb{R}^3$ denotes translation from the origin of the inertial frame to the origin of the camera frame. Now, if the target position in the inertial frame is known, its position in camera center frame would be obtained from the knowledge of the camera center position and coordinate transforms. If the position of the target in inertial and camera frame is described as $\vec{d}_t^i$ and $\vec{d}_t^c$, then the relation can be described as

$$\vec{d}_t^c = [T_c^i]^{-1} \vec{d}_t^i \quad (3)$$

Once the target position is determined relative to the camera center, it can be transformed to gimbal frame. Then the gimbal is maneuvered to point the camera towards the object so that the object moves to the center of the image frame. Pointing accuracy can be improved by predicting the state of the camera and the target [50]. However, the target in our system is non-cooperative and it does not provide its position information. As a result, a coordinate transform will not provide target state estimation with respect to the camera frame.

The other method involves estimating the state using the information obtained from the camera mounted on the UAV. In order for the estimation with the camera, elevation information of the ground plane is needed. A known terrain model would provide the elevation information for the estimation and the knowledge of the location and orientation of the UAV and its camera system can be used for target estimation. The system described in [5] estimates the non-cooperative target state in the absence of a terrain model, where a flat earth model is assumed. However, the estimation accuracy depends largely on the assumption accuracy and will generate erroneous estimation if the terrain is not flat.

Accuracy of the state estimation and prediction depends largely on the sensor information accuracy and uncertainties in environment. Any change in atmosphere can affect the GPS radio signals. Wind causes the course (direction of

travel) of the aircraft to deviate from the heading (direction of the x-axis in the aircraft body fixed coordinate system). As a result actual velocity of the airplane differs significantly in the presence of strong wind. Since the camera with the gimbal is mounted to the body, the difference between course and heading, if not accounted for, will cause significant error in the estimations. A comprehensive description of the uncertainties in geo-location estimation errors is presented in [5, 6]. They have shown a relation between the UAV altitude and geo-location estimation error. As altitude increases, a fixed error associated with the UAV roll angle increases, thus produces a localization error. At low altitude high oblique angle produce a larger localization error. An optimal altitude and orbit radius are also proposed that minimizes the geo-location estimation error. However, this approach limits the UAV functionality. For these reasons, an adaptive neuro-fuzzy based gimbal control algorithm is proposed in this paper.

The proposed algorithm offers an innovative method of gimbal motion generation without an exact mathematical model and in presence of uncertainties in the relative velocity estimation of the object. The controller parameters are updated from errors to generate variable speed and position commands for the motion controller.

2.3 Super Hauler UAV

A customized UAV named “Super Hauler” operated by the UASE (Unmanned Aerial System Engineering) lab, at the University of North Dakota houses the payload as well as the Piccolo autopilot from CloudCap Technology with its dedicated control link. The UAV has a wingspan of 3.66 m, a wing area of 2.374 m², and 3.05 m of length. The UAV weighs 21.8 kg and has 11.34 kg of payload carrying capacity. Figure 3 shows the “Super Hauler” UAV.

2.4 SUND OG Payload

The onboard visual tracking system has been implemented on the SUND OG (Surveillance by University of North Dakota Observational



Fig. 3 BTE Super Hauler UAV operated by the UASE for payload flight testing

Gimbal) payload developed by the UASE lab at the University of North Dakota. The payload consists of an optical camera mounted on a three axes gimbal system. The SUND OG payload is shown in Fig. 4. The gimbal has 360° rotation and +30° pan and tilt motion. A detail description of the Payload and its functionality can be found in [50, 51]. The third axis of the gimbal is used for orienting the image. Since we are only interested in keeping the object at the center of the image frame without considering its orientation, the neuro-fuzzy controller has been implemented on the pan and tilt axes of the gimbal.

3 Neuro-Fuzzy Controller Architecture

3.1 Fuzzy System and Fuzzy If-Then Rules

The fuzzy inference system is used to derive conclusion from a set of fuzzy if-then rules and one or more conditions. The “if” part of the if-then rule is called the premise part and the “then” is called consequence. Takagi and Sugeno [9] presented a mathematical model (T-S model) of fuzzy if-then rule, where fuzzy sets are involved only in the premise part and the consequent part is described by non fuzzy equations of the input variables.

Rule 1: If x_1 is A_1 and x_2 is B_2 , then $f_1 = a_1x_1 + b_1x_2 + c_1$

Rule 2: If x_1 is A_2 and x_2 is B_2 , then $f_2 = a_2x_1 + b_2x_2 + c_2$

where the firing strength (or weight) of the i th rule is obtained as the sum of the membership values on the premise part.

3.2 Controller Architecture

A RBFN based Neuro-fuzzy system has been developed to control the pan and tilt motions of the camera in our vision based tracking system in order to keep the tracked object always centered in the image frame. If the object moves fast from one place to another in successive image frames, the camera with the gimbal, needs to be move at the same speed to keep the object in the middle of the image frame. On the other hand when the object moves slowly we also would like the camera to move slowly. Examples of the basic fuzzy if-then rules for our system:

- Rule 1: if the x position is far from the center of the image and if distance from the center is greater than the distance in the previous frame then camera movement and velocity in the x coordinate towards the center is large
- Rule 2: if the x position is close to the center of the image and if distance from the center is smaller than the distance from the center in the previous frame then camera movement and velocity in the x coordinate towards the center is small. The full set of rules is explained in a tabular form in the implementation section.

Two separate but identical neuro-fuzzy systems have been used to control the two axes motions of the gimbal. Each system has two inputs, detected position and difference in positions in two successive frames in one axis of the image frame. Outputs are the generated position and velocity commands for the gimbal motion controller in each axis. The neuro-fuzzy system architectures for both axes are identical. One of them is described in this section. A basic architecture of the neuro-fuzzy systems is shown in Fig. 5.

The first layer of the structure is the radial basis function layer where fuzzification of the crisp inputs x_1 and x_2 takes place. This means that the degree of membership of the inputs x_1 and x_2 are calculated for each of the corresponding fuzzy

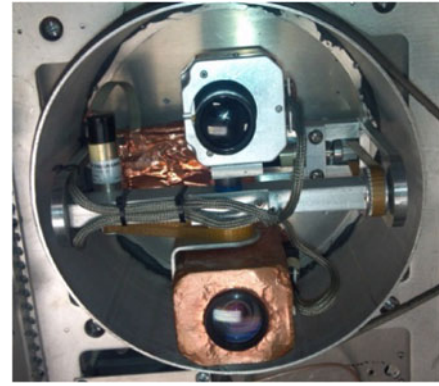


Fig. 4 SUNDG (Surveillance by University of North Dakota Observational Gimbal) payload. Image shows the three-axis gimbal system with the Electro-Optical (EO) and Infrared (IR) cameras mounted on it

sets A_{ij} . The membership functions are Gaussian functions and each membership function has its own center and width. Figure 4 shows the internal structure of the j th neuron in the 1st layer where the input vector is $\vec{x} = [x_1, x_2]$, $\vec{\mu}_j = [\mu_{1j}, \mu_{2j}]$ is the vector of membership functions in the j th neuron, $\vec{C}_j = [c_{1j}, c_{2j}]$ is the vector of centers and $\vec{\sigma}_j = [\sigma_{1j}, \sigma_{2j}]$ is the width vector for the membership functions in the j th neuron. The membership functions are expressed as

$$\mu_{ij} = \exp \left[-\frac{(x_i - c_{ij})^2}{2\sigma_{ij}^2} \right], \quad i \in \{1, 2\} \quad \text{and} \quad j \in \{1, 2, 3, \dots, 49\} \quad (4)$$

where each neuron in this layer represents an if-part of a fuzzy rule. The output of the neuron is computed by the sum of grades of membership functions. The output of the j th neuron is:

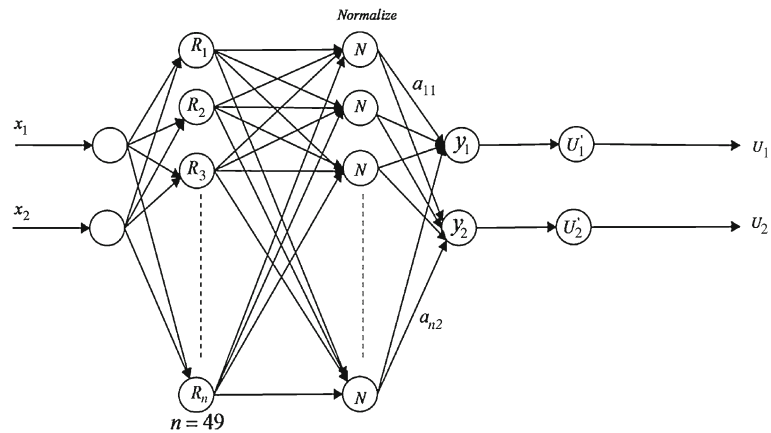
$$R_j = \exp \left[-\sum_{i=1}^2 \mu_{ij} \right], \quad j \in \{1, 2, \dots, 49\} \quad (5)$$

Internal structure of the j th neuron is shown in Fig. 6.

Layer 2 is the normalized layer. The rule firing strength should also be taken as a measure of the degree of compliance of the current input state with a rule in the rule base, taking the value 1 when compliance is total.

Layer 3 is the consequence part of our RBF based fuzzy system and contains a set of adjustable parameters to learn. We have adapted our fuzzy

Fig. 5 Basic architecture of our neuro-fuzzy system. Input x_1 and x_2 are the detected object distance from the center and difference in distance between successive frames. The outputs are the position and velocity commands for the motion controller



inference system where the consequence is a first-order linear equation based on Takagi-Sugeno [9] model with a crisp output.

Rule I: if (x_1 is A_{1i}) andand (x_n is A_{ni}) then

(y_1 is $f_{i1}(x)$) and and (y_p is $f_{ip}(x)$),

where $f_{ik}(x) = a_{ik0} + a_{ik1}x_1 + a_{ik2}x_2 + \dots + a_{ikn}x_n$ and $a_{ik0}, a_{ik1}, a_{ik2}, \dots, a_{ikn}$ are connection weights.

The output of the k th element of layer 3 is given by

$$y_k = \frac{\sum_{j=1}^{49} R_j(\vec{x}) f_{jk}(\vec{x})}{\sum_{j=1}^{49} R_j(\vec{x})}, k \in \{1, 2\} \quad (6)$$

Layer 4 is the output layer and has 2 nodes for position and velocity control of the gimbal. A Sigmoid function (S function) has been employed as the nodes output transfer functions. The Sigmoid function has characteristics of non-linearity and

differentiability and the gradient is steeper at middle part and flatter at two ends thus enhancing the non-linear map ability of neural network. The S function is shown in Fig. 7. The k th element of the output from layer 4 is defined as

$$U'_k = \frac{1}{1 + \exp(-y_k)} \quad (7)$$

where y_k is the k th element from the layer 3 of the network.

As the output $[0, 1]$ of S function is not in the range of the output variable from the neuro fuzzy controller to drive the motion controller, the linear transformation should be done as follows:

$$U_k = U'_k (U_{k \max} - U_{k \min}) + U_{k \min} \quad (8)$$

where $U_{k \max}$ and $U_{k \min}$ are the maximum and minimum values for the inputs to the motion controller. $\vec{U}$ is the final output vector of the system in position and velocity commands.

4 Implementation

4.1 Neuro-Fuzzy Controller

Implementation of the RBFN based neuro-fuzzy controller is presented here. The object position in the image frame is detected by the onboard visual tracking system using zero mean normalized cross correlation described in the previous section. We get the pixel position of the detected object in the image coordinate system. Now the input to each

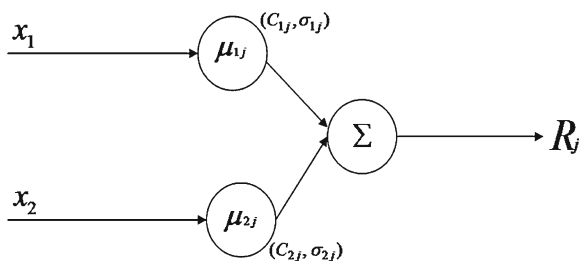


Fig. 6 Internal structure of a neuron, each neuron represents one rule for the fuzzy inference system

Table 1 Initial fuzzy rules for the neuro-fuzzy controller

		Position of the object in image frame						
		NB	NM	NS	0	PS	PM	PB
Difference in position in two successive frames	NB	f ₁	f ₂	f ₃	f ₄	f ₅	f ₆	f ₇
	NM	f ₈	f ₉	f ₁₀	f ₁₁	f ₁₂	f ₁₃	f ₁₄
	NS	f ₁₅	f ₁₆	f ₁₇	f ₁₈	f ₁₉	f ₂₀	f ₂₁
	0	f ₂₂	f ₂₃	f ₂₄	f ₂₅	f ₂₆	f ₂₇	f ₂₈
	PS	f ₂₉	f ₃₀	f ₃₁	f ₃₂	f ₃₃	f ₃₄	f ₃₅
	PM	f ₃₆	f ₃₇	f ₃₈	f ₃₉	f ₄₀	f ₄₁	f ₄₂
	PB	f ₄₃	f ₄₄	f ₄₅	f ₄₆	f ₄₇	f ₄₈	f ₄₉

neuro-fuzzy system includes the distance of the detected object ($\vec{x}_1$) and the difference in distance ($\vec{x}_2$) of the object from the center of the image in two successive frames. If the center position in the image frame is denoted as $\vec{o} = [o_x, o_y]$ and the detected position of the object in k th frame is denoted as $\vec{u}$ (u_k, v_k) then the inputs to the pan and tilt motion controllers are expressed as

$$\begin{aligned}\vec{x}_{k1} &= \vec{u}_k - \vec{o}; \\ \vec{x}_{k2} &= \vec{u}_k - \vec{u}_{k-1};\end{aligned}\quad (9)$$

The output variables from the neuro-fuzzy systems are pan and tilt position and velocity commands for the gimbal motion controller. As mentioned earlier, two separate but identical neuro-fuzzy systems have been implemented for two axes motion of the gimbal. Each system is initialized with 49 neurons. Each neuron consists of two radial basis functions for distance and difference in distance with center vector $\vec{C} = [c_1, c_2]$ and width vector $\vec{\sigma} = [\sigma_1, \sigma_2]$. Each neuron corresponds to a fuzzy rule. Both the distance and difference in distance are partitioned into 7 receptive field units with center and width vectors. These receptive fields are equivalent to fuzzy sets: Negative Big (NB), Negative Medium (NM), Negative Small (NS), Zero (Z), Positive Small (PS), Positive Medium (PM) and Positive Big (PB).

A first order Takagi-Sugeno [9] method has been adopted as the consequence part of the fuzzy system. The premise variable is still expressed by linguistic terms while the result variable is expressed by an exact value. The fuzzy rules are shown in Table 1.

Both the premise and consequence parts contain a set of adjustable parameters to learn. The center and width vectors for the neurons and the

connection weights are updated with the learning algorithm during training. The schematic diagram of the controller during training period is shown in Fig. 8.

4.2 Learning Algorithm

Learning in the RBFN based Neuro Fuzzy controller consists of adjusting the mean ($\vec{C}$) and variance ($\vec{\sigma}$) vectors of individual hidden nodes as well as their weights (A) matrix for each system. Initially a fixed number of hidden nodes are chosen based on our prior knowledge. The mean vector $\vec{C}$ for each neuron along with the width vector $\vec{\sigma}$ is also chosen. Then the learning algorithm incrementally updates the parameters ($\vec{C}$, $\vec{\sigma}$ and A) based on the current training pattern. This serves to allocate network resources efficiently by placing the domains of membership functions covering only those regions of input/output space where data are present. The network parameters are updated by applying the gradient descent

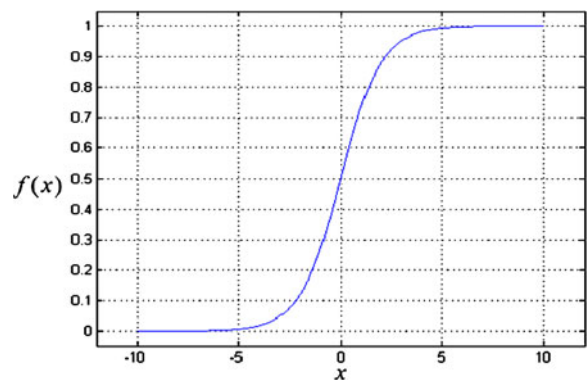
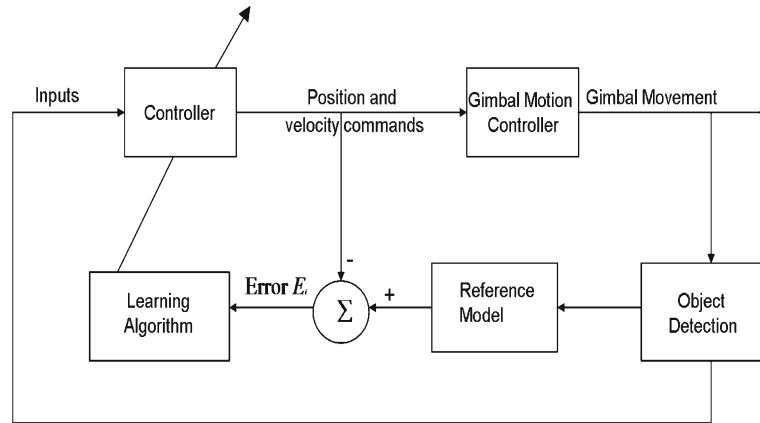
**Fig. 7** A sigmoid curve. The output ranges from 0 to 1

Fig. 8 A schematic diagram of the neuro-fuzzy based controller during training period. Error in position and velocity are fed back to the controller to update mean, variance vectors and weight



method to an error function. The error, E_i , for the i th training pattern is defined as:

$$E_i = \frac{1}{2} \sum_{k=1}^p (t_{ik} - t_{ack})^2, \quad (10)$$

where p represents the number of output units, t_{ik} represents the k th element of the desired output vector for i th teaching pattern, and t_{ack} represents the k th element of the actual output vector for the i th training pattern.

The central part of learning rule for the neural network using gradient descent method is to recursively obtain a gradient vector in which each element is defined as the derivative of an error measure with respect to a parameter. This is done by means of the chain rule, known as the back propagation learning rule where the gradient vector is calculated in the direction opposite to the flow of the output of each node.

4.3 Derivation of the Parameter Learning

4.3.1 Weight Update

To update the connection weight, the derivative of the error measure with respect to the connection weight is computed. Update rules for the connection weight a_{ikj} in i th iteration is

$$a_{ikj}^{new} = a_{ikj}^{old} + \eta \Delta a_{ikj} = a_{ikj}^{old} - \eta \frac{\partial E_i}{\partial a_{ikj}} \quad (11)$$

where a_{ikj} is the connection weight between hidden neuron j and output neuron k in i th pattern. η is the learning rate of the system. A slower learning rate means error takes much time to converge but has a less chance of being trapped into local minima.

There is no prior knowledge about the desired position and velocity commands for the motion controller to keep the object at the center of the image frame i.e. how much the gimbal should move to keep the object at the center. As a result the error cannot be calculated from the desired and actual position and velocity commands for the motion controller. However, we can calculate the position and velocity error from the object detection in the image frame. The positional error is the difference between the center of the image frame and the actual position of the object in the image frame detected by the tracking system. The velocity error is the difference between the desired and actual velocity of the detected object in the image frame. The desired velocity is defined as the difference in pixel position required to move the object to the center in two successive image frames. The actual velocity is the actual positional difference of the detected object in two successive image frames. The error is calculated as the square sum of the position and velocity error. The desired position is the center of the image frame, which in this case are 160 in the horizontal pixel position and 120 in the vertical pixel position.

Using the back propagation method, the gradient Δa_{ijk} can be defined as:

$$\Delta a_{ijk} = -\frac{\partial E_i}{\partial a_{ijk}} = -\frac{\partial E_i}{\partial t_{iack}} \frac{\partial t_{iack}}{\partial U_{ik}} \frac{\partial U_{ik}}{\partial U'_{ik}} \frac{\partial U'_{ik}}{\partial y_{ik}} \frac{\partial y_{ik}}{\partial a_{ijk}} \quad (12)$$

Each part of the derivation is explained below.

The first term $\frac{\partial E_i}{\partial t_{iack}}$ represents the derivative of the error with respect to the actual position or velocity detected by the tracking system, as we have the detected position and velocity on the image frame as our measurements. The derivative $\frac{\partial E_i}{\partial t_{iack}}$ can be expressed as

$$\frac{\partial E_i}{\partial t_{iack}} = (t_{ik} - t_{iack}) \quad (13)$$

The second term $\frac{\partial t_{iack}}{\partial U_{ik}}$ represents the derivative of the measurement with respect to the output of the neuro-fuzzy controller. This term is unknown. However we can assume $\frac{\partial t_{iack}}{\partial U_{ik}} \approx \frac{\Delta t_{iack}}{\Delta U_{ik}}$ which is the difference in position over the difference in motor count. As we know that the actual output of the neuro-fuzzy controller is the linear transformation of the output from the sigmoid function, the term $\frac{\partial U_{ik}}{\partial U'_{ik}}$ is the derivative of the actual output with respect to the output of the sigmoid function, which is expressed as:

$$\frac{\partial U_{ik}}{\partial U'_{ik}} = (U_{k \max} - U_{k \min}) \quad (14)$$

Output of the system is defined as S function:

$$U'_{ik} = \frac{1}{1 + \exp(-y_{ik})} \quad (15)$$

The term $\frac{\partial U'_{ik}}{\partial y_{ik}}$ is expressed as

$$\begin{aligned} \frac{\partial U'_{ik}}{\partial y_{ik}} &= \left(1 - \frac{1}{1 + \exp(-y_{ik})}\right) \frac{1}{1 + \exp(-y_{ik})} \\ &= (1 - U'_{ik}) U'_{ik} \end{aligned} \quad (16)$$

If the consequence is first order linear equation then output from k th node y_k is described as

$$y_k = \frac{\sum_{j=1}^{49} R_j(\vec{x}) f_{jk}(\vec{x})}{\sum_{j=1}^{49} R_j(\vec{x})} \quad (17)$$

with

$$f_{jk}(\vec{x}) = a_{jk0} + a_{jk1}x_1 + a_{jk2}x_2 \quad (18)$$

The final term $\frac{\partial y_{ik}}{\partial a_{ijk}}$ is described as

$$\frac{\partial y_{ik}}{\partial a_{ijk}} = \frac{R_j}{Z} x_n \quad n \in \{1, 2\} \quad (19)$$

where

R_j = output of the j th neuron

$Z = \sum_{j=1}^{49} R_j(x)$ = Sum of the output from the hidden neurons and

Finally with all the terms together the gradient is expressed as

$$\begin{aligned} \Delta a_{ijk} &= -(t_{nk} - t_{iack}) * \frac{\Delta t_{iack}}{\Delta U_{ik}} * (U_{k \max} - U_{k \min}) \\ &\quad * (1 - U'_{ik}) U'_{ik} * \frac{R_j}{Z} x_n \end{aligned} \quad (20)$$

similarly center and variance vectors can be updated using gradient descent algorithm.

4.4 Learning Process Flow

The detected position of the object in the image frame is used to compute position and velocity errors for the controller. The squared sum of errors is then fed back to the controller to update the controller parameters. The detected position in the current frame and difference in position between the current and previous frames are the inputs for the neuro-fuzzy controller. The controller generates position and velocity command for the gimbal. The object is detected in a new position in the next frame and the detected position is used again to calculate position and velocity errors for the controller. The learning process flows for the neuro-fuzzy systems are identical for both pan and tilt motions. The process flow for the pan motion is explained as follows.

The first step starts with the image frame where the object is detected for the first time after the patch has been extracted and the system starts to track. Object detection process has been described in the previous section. Once the object is detected, its position in the image coordinate is obtained. Now the input to the controller at

step 1 is $\vec{x}_1 = (x_{11}, x_{12})$ where x_{11} is the detected position of the object and $x_{12} = (x_{11} - 0)$ is the difference in position in two successive frames. The controller output is used to move the gimbal. In this frame the weights of the membership functions are not updated.

In step 2, the new position x_{21} is obtained from the detection algorithm and the velocity $(x_{21} - x_{11})/t$ of the object moving in the image frame is computed. Error is calculated from the difference between desired position and measured position and between desired velocity and measured velocity. This error is then used to update the parameters previously described. Now the new position x_{21} and difference in position between current and previous frame $x_{22} = (x_{21} - x_{11})$ is the new input $_2 = (x_{21}, x_{22})$ to the controller. Output from the controller in the form of motor count is then sent to the motor controller to move the gimbal. The process continues in the subsequent frames.

5 Testing and Results

5.1 Training of Neuro-fuzzy System

The neuro-fuzzy system was trained with the learning algorithm described in Section 4. The

object was detected in each frame using our visual tracking system which is explained in Section 2. Position of the detected object and difference in position are the inputs for our neuro-fuzzy system. The system then generates position and velocity commands for the gimbal motion controller. The camera with the gimbal moves and the object is detected in a new position in the image frame. Position error is calculated from the difference between desired position which is the center of the image frame (in our case 160, 120) and actual position in the image frame. The velocity error is calculated from the difference between desired velocity ($\Delta x/t$) and actual velocity. The error is then fed back to the controller to update the parameters of the network. Two different objects were selected for the training of the neuro-fuzzy controller. The objects were located at different positions in the image frame. The system detects one object in the image frame and generates position and velocity command for the gimbal to move the object to the image center. After the controller stabilizes, the other object is selected and gimbal is moved to the new position to move the other object to the image center. These processes continue until the error in generating position and velocity commands falls below a pre-defined threshold.

Figure 9 shows the training of the neuro-fuzzy controller for both pan and tilt motion of the

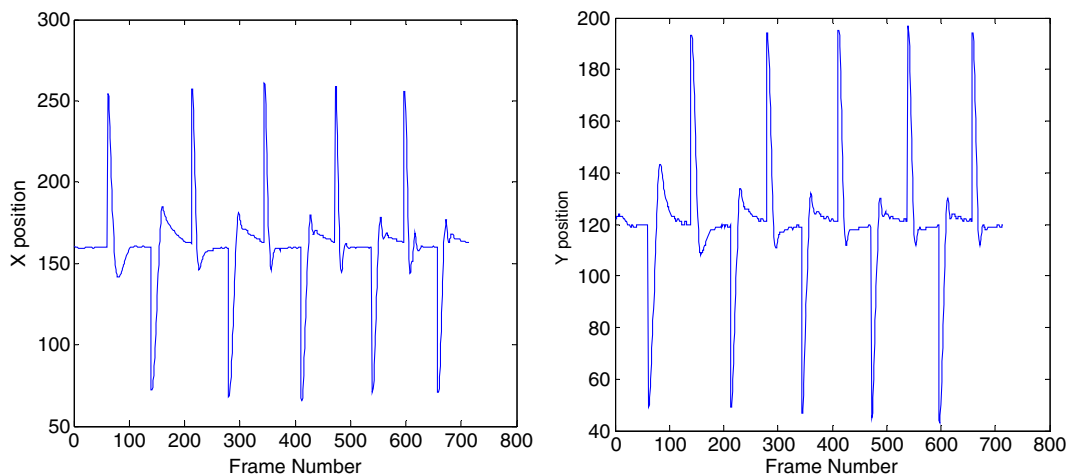


Fig. 9 Training of the neuro-fuzzy system for motion in x and y direction on the image frame. The controller is trained with selecting two different objects in the image frame. The images show that with time the controller up-

dates the network and improves performance with each iteration and generates accurate position command for the gimbal motion controller

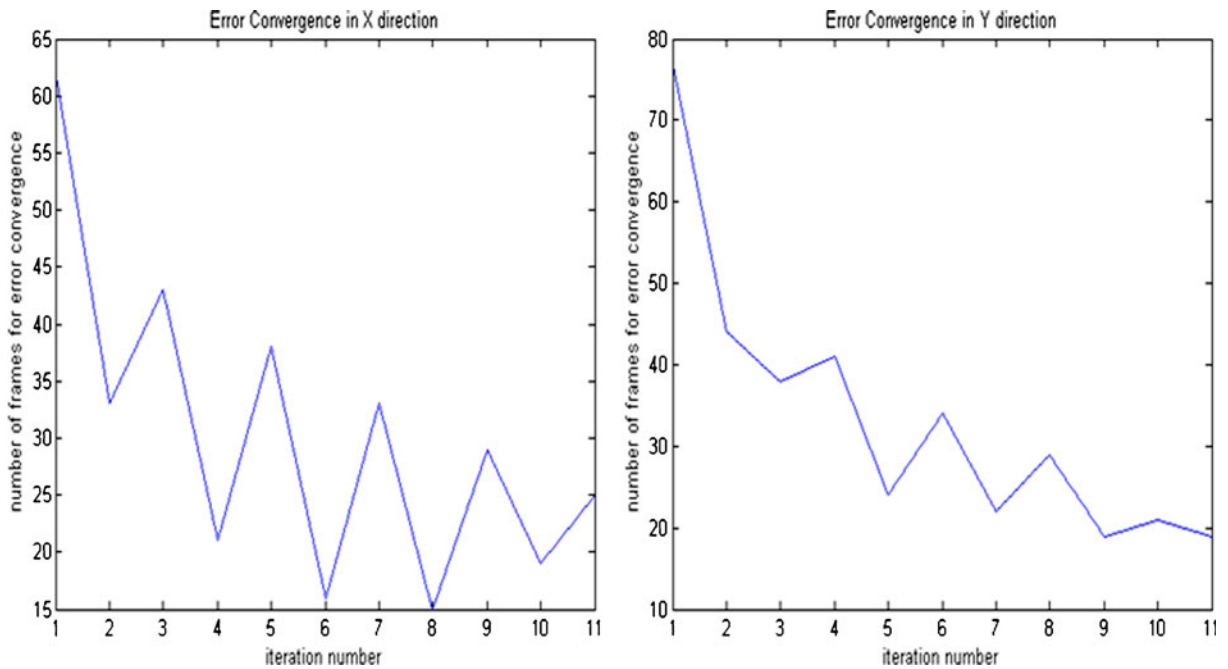


Fig. 10 Convergence of error for x and y axes during training. Oscillatory pattern arises due to alternately selecting two different objects. Number of frames required for error convergence reduced with each iteration

gimbal. The graphs show the positions of the two objects in the image frame. The first object is selected in the image frame and the gimbal moves to

keep the object at the center. Then the other object is selected and the gimbal moves to a new position to keep the new object at the image center.

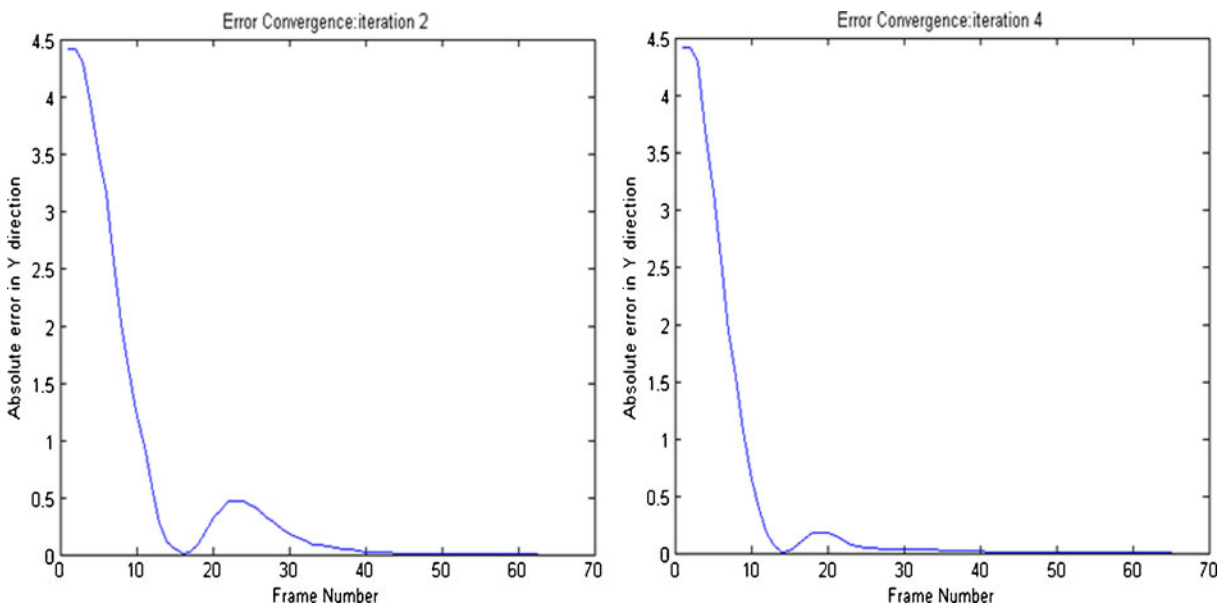


Fig. 11 Convergence of error during training. The controller shows improved performance with training and stabilizes quickly

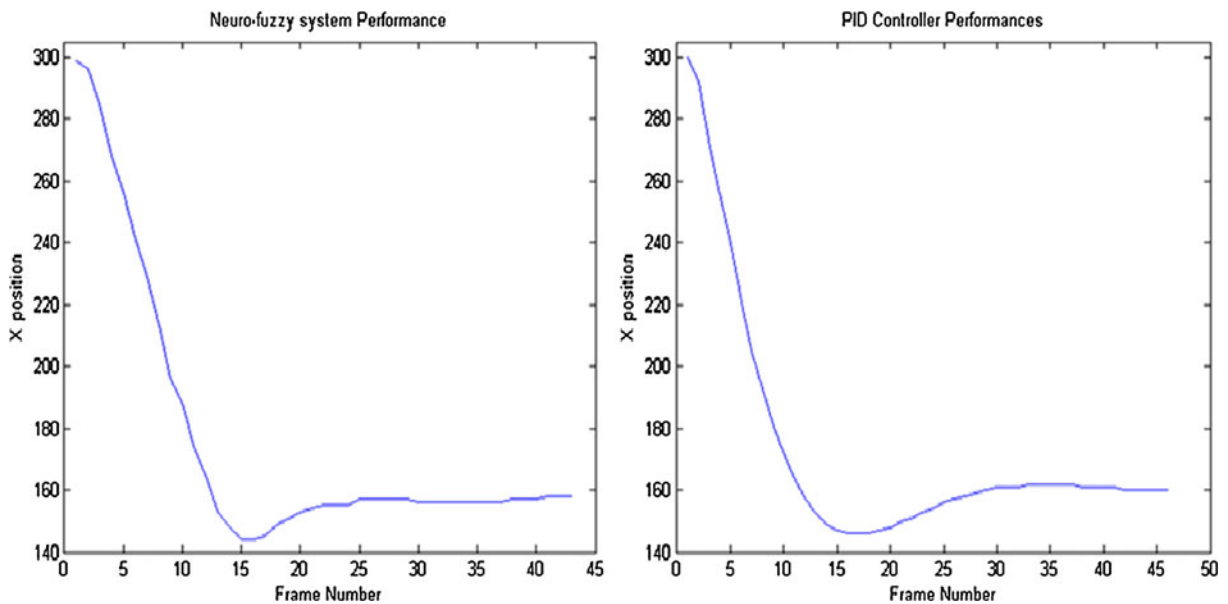


Fig. 12 Performances comparison between the neuro-fuzzy controller and the PID controller

Initially the neuro-fuzzy system generates position and velocity commands that are inaccurate and takes longer time to move the gimbal to the correct position. Then the controller is trained with the measured data and it can be seen that with each iteration of training, the controller generates more accurate position and velocity commands for the gimbal motion controller to keep the objects at the center of image frame and stabilizes faster. It should be noted that the center of the image is pixel position 160 in X-axis and pixel position 120 in Y-axis.

Figure 10 shows the error convergence during training. Initially the network generates inaccurate position and velocity commands and it takes more time to converge. The objects were located at different distances from the center of the image frame. Objects close to the center of the frame required less time to converge compare to the objects relatively far from the center. An oscillatory pattern of convergence was generated since the two objects were selected alternately. However, with each iteration, the number of frames required for error convergence reduced. The controller generates more accurate position and velocity commands to move the gimbal in order to keep the object at the center of image frame and error converges very quickly.

Figure 11 shows a closer look at the error convergence. The left image shows the error convergence during iteration 2. The right image shows error convergence during iteration 4. At the beginning of EACH iteration, the selected object is far from the center of the image frame and the error is large. Then the gimbal moves to keep the object at the center. During iteration 2, the system takes 37 frames for the error to go below the threshold 0.05 and stabilize with a frame rate about 25 frames per second. The controller performance improves with training which is shown in the right image. During iteration 4, the controller converges faster and takes 25 frames for the error to go below 0.05 and stabilize.

The Neuro-Fuzzy controller performances have been compared with the classical PID controller in the lab environment. A set of stationary objects has been selected for tracking and the controllers generate commands for the gimbal motion controller in order to move the objects to the center of the image frame. For the PID controller, an optimal set of controller gains were selected empirically that reduced the overshoot while moved the object to the center in shortest possible time. Result of the performance comparison is shown in Fig. 12. The result shows the object trajectory generation for

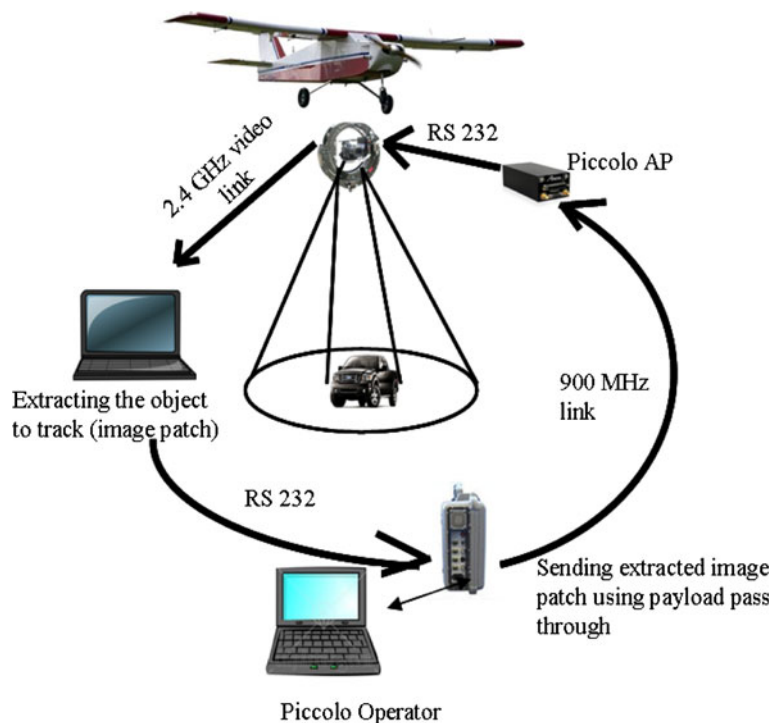
both controllers and the Neuro-Fuzzy controller outperforms the PID controller in terms of time required to move the object to the center. The PID controller took about 30 frames to move the object to the center of the image frame whereas it took 22 frames for the Neuro-Fuzzy controller to reach the object to the center.

5.2 Flight Test

Actual flight tests were conducted during May–August, 2011 in order to verify the controller algorithm. The flight tests validate the algorithm and shows that the tracking system is capable of generating accurate position and velocity command for the gimbal motor controller in order to keep the object at the center of the image frame. The testbed for the tracking system is explained in Fig. 13. Initially the controller was trained off-line before the flight. The UAV was flying at 600 feet altitude and a repeated flight path over the ground control tent was chosen for the UAV. The video captured by the payload camera was broadcast to the ground computer using a 2.4 GHz communication link. On the ground the interest

object was selected from the video and sent to the UAV during one pass over the object. The object was lost when it went out of the camera field of view as the UAV flew away from the target. Once the UAV came over the object again in the next pass, it detected the object and started tracking. Then the controller generates position and velocity commands for the gimbal. The gimbal with the camera moved accordingly to keep the object at the center of the image frame. The training of the neuro-fuzzy controller was carried on during the flight to make the controller adaptive. It should be noted that the parameters and connection weights were not updated when there was no detection of the object and the gimbal went to its home position when the object was not detected in 15 consecutive frames. The tracking system successfully tracked different objects during the flight tests. The results of the flight test have been shown in Fig. 14. The object was detected in frame 1,514 and the gimbal was moved to keep the object at the center of the frame. In frame 1,526, the object was not detected. As a result the gimbal was not moved and the Neuro-fuzzy network connections were not updated. However, the object was

Fig. 13 Flight test bed. The object is selected on the ground computer and then sent to the UAV using payload pass-through of Piccolo autopilot



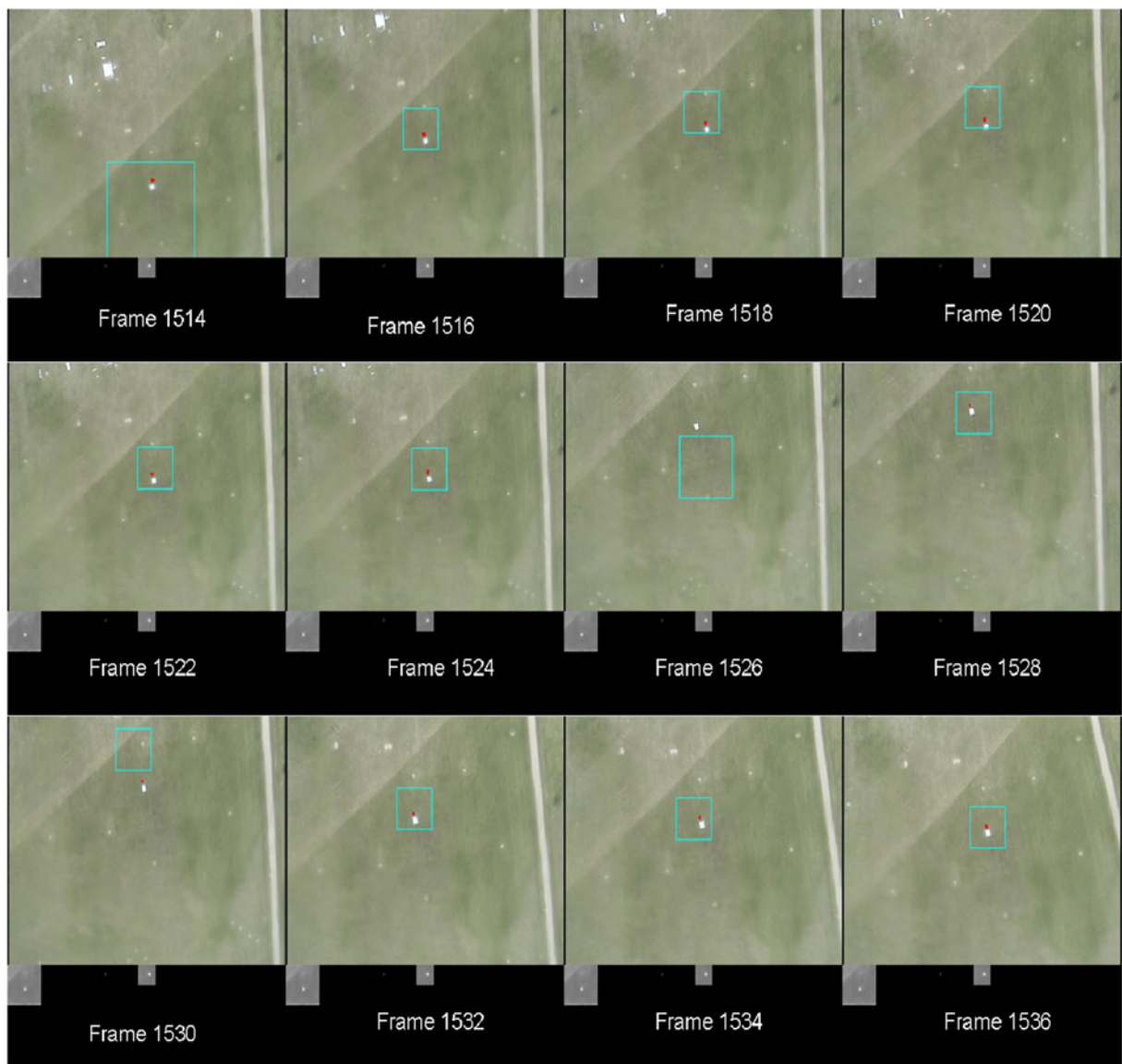


Fig. 14 Flight test results. The object was detected in one image frame and then the gimbal was actuated to keep the object in the center of the image frame. Detection failed in

one frame as the object went out side the search window. However, once the object was re-detected, the gimbal was moved to keep the object at the center of the image frame

detected again in the next frame and the gimbal was moved and the object was kept at the center of the image frame.

6 Conclusions

A vision based neuro-fuzzy motion controller for the gimbal movement has been developed

considering non-linearity of the tracking process. The input variables of the neuro-fuzzy controller are the detected position of the object of interest in the image frame and difference in position between two successive image frames. The output variables to the gimbal motion controller are position and velocity commands for the pan-tilt motion of the gimbal. The tracking algorithm detects the object in image frames. The detected position

and difference in positions are then sent to the neuro-fuzzy controller as inputs. The controller generates position and velocity commands for the gimbal motion controller in order to move the gimbal. The gimbal moves accordingly to keep the detected object at the center of the image frame. An online learning method has been developed using gradient descent method to train the network in real time. The network can be trained for a fixed number of iterations or by assigning an error threshold value. Results show that the neuro-fuzzy control system can learn from the error and generate position and velocity commands for the motion controller. It does this without knowing the exact mathematical relationship between the velocity counts of the motion controller and the position and velocity of the object in the image frame. The system is initialized with 49 neurons with mean and variance vectors. Then parameters of the system (mean, variance vector and connection weight) for the neurons are updated online with the training data. The system can be optimized with extraction of fuzzy rules during training of the network. Experimental result shows that the system learns from errors during training and generates accurate position and velocity commands once trained.

Distance travelled by an object in world or camera coordinate system appears different in image coordinate system depending on the position of the object to the camera. A projective transformation from image coordinate frame to camera coordinate frame incorporating the position of the object in the image frame and projective distortion will make the input more accurate for the neuro-fuzzy system. An online extraction of neurons during the training of the neuro-fuzzy system can be employed to make the system more autonomous.

Acknowledgements This research was supported in part by Department of Defense contract number FA4861-06-C-C006, "Unmanned Aerial System Remote Sense and Avoid System and Advanced Payload Analysis and Investigation," and North Dakota Department of Commerce grant entitled "UND Center of Excellence for UAV and Simulation Applications." The authors appreciate the contributions of the Unmanned Aircraft Systems Engineering (UASE) Laboratory team at the University of North Dakota.

References

1. Kumar, R., Samarasekera, S., Hsu, S., Hanna, K.: Registration of highly-oblique and zoomed in aerial video to reference imagery. In: Proceedings of IEEE Computer Society Computer Vision and Pattern Recognition Conference. Barcelona, Spain (2000)
2. Kumar, R., Sawhney, H.S., Asmuth, J.C., Pope, A., Hsu, S.: Registration of video to geo-referenced imagery. In: Proceedings of the 14th International Conference on Pattern Recognition, vol. 2, pp. 1393–1400. Brisbane, Australia (1998)
3. Stolle, S., Rysdyk, R.: Flight path following guidance for unmanned air vehicles with pan-tilt camera for target observation. In: 22nd Digital Avionics Systems Conference, Indianapolis (2003)
4. Dobrokhodov, V.N., Kaminer, I.I., Jones, K.D., Ghabcheloo, R.: Vision-based tracking and motion estimation for moving target using small UAVs. In: Proceedings of 2006 American Control Conference, Minneapolis, 14–16 June 2006
5. Barber, D.B., Redding, J.D., McLain, T.W., Beard, R.W., Taylor, C.N.: Vision-based target geo-location using a fixed-wing miniature air vehicle. *J. Intell. Robot. Syst.* **47**, 361–382 (2006)
6. Redding, J.D.: Vision based target localization from a small fixed-wing unmanned air vehicle. Master's thesis, Brigham Young University, Provo, Utah 84602 (2005)
7. Qadir, A., Neubert, J., Semke, W.: On-board visual tracking with small unmanned aircraft systems. In: AIAA Infotech@Aerospace conference, St. Louis, MO, 28–31 March 2011
8. Jang, J.S.R.: ANFIS: adaptive-network-based fuzzy inference system. *IEEE Trans. Syst. Man Cybern.* **23**, 665–684 (1993)
9. Takagi, T., Sugeno, M.: Fuzzy identification of systems and its applications to modeling and control. *IEEE Trans. Syst. Man Cybern.* **15**, 116–132 (1985)
10. Leng, G., McGinnity, T.M., Prasad, G.: An approach for on-line extraction of fuzzy rules using a self-organizing fuzzy neural network. *Fuzzy Sets and Systems* **150**, 211–243 (2005)
11. Li, W., Hori, Y.: An algorithm for extracting fuzzy rules based on RBF neural network. *IEEE Trans. Ind. Electron.* **53**(4), 1269–1276 (2006)
12. Jang, J.S.R., Sun, C.T.: Functional equivalence between radial basis function networks and fuzzy inference systems. *IEEE Trans. Neural Netw.* **4**, 156–159 (1993)
13. Cho, K.B., Wang, B.H.: Radial basis function based adaptive fuzzy systems and their applications to system identification and prediction. *Fuzzy Sets and Systems* **83**, 325–339 (1996)
14. Diao, Y., Passino, K.M.: Adaptive neural/fuzzy control for interpolated nonlinear systems. *IEEE Trans. Fuzzy Syst.* **10**, 583–595 (2002)
15. Chen, B.S., Lee, C.H., Chang, Y.C.: Tracking design of uncertain nonlinear siso systems: adaptive fuzzy approach. *IEEE Trans. Fuzzy Syst.* **4**, 32–43 (1996)

16. Spooner, J.T., Passino, K.M.: Stable adaptive control using fuzzy systems and neural networks. *IEEE Trans. Fuzzy Syst.* **4**, 339–359 (1996)
17. Lee, C., Teng, C.: Identification and control of dynamic systems using recurrent fuzzy neural networks. *IEEE Trans. Fuzzy Syst.* **8**, 349–366 (2000)
18. Narendra, K.S., Parthasarathy, K.: Identification and control of dynamical systems using neural networks. *IEEE Trans. Neural Netw.* **1**, 4–27 (1990)
19. Polycarpou, M.M., Mears, M.J.: Stable adaptive tracking of uncertain systems using nonlinearly parameterized online approximators. *Int. J. Control* **70**(3), 363–384 (1998)
20. Hsiao, F.-H., Xu, S.-D., Lin, C.-Y., Tsai, Z.-R.: Robustness design of fuzzy control for nonlinear multiple time-delay large-scale systems via neuralnetwork-based approach. *IEEE Trans. Syst. Man Cybern. B Cybern.* **38**(1), 244–251 (2008)
21. Lewis, F.L., Jagannathan, S., Yildirek, A.: *Neural Network Control of Robot Manipulators and Nonlinear Systems*. Taylor and Francis, London (1999)
22. Golea, N., Golea, A., Benmahammed, K.: Stable indirect fuzzy adaptive control. *Fuzzy Sets and Systems* **13**(7), 353–366 (2003)
23. Wang, T., Lina, C., Liub, H.: Observer-based indirect adaptive fuzzy-neural tracking control for nonlinear SISO systems using VSS and H^∞ approaches. *Fuzzy Sets and Systems* **14**(3), 211–232 (2004)
24. Labiod, S., Boucherit, M.S., Guerra, T.M.: Adaptive fuzzy control of a class of MIMO nonlinear systems. *Fuzzy Sets and Systems* **15**(1), 59–77 (2005)
25. Rovithakis, G.A., Christodoulou, M.A.: Adaptive control of unknown plants using dynamical neural networks. *IEEE Trans. Syst. Man Cybern.* **24**, 400–412 (1994)
26. Chen, F.C., Liu, C.C.: Adaptively controlling nonlinear continuous-time systems using multilayer neural networks. *IEEE Trans. Automat. Control* **39**, 1306–1310 (1994)
27. Rovithakis, G.A., Christodoulou, M.A.: Direct adaptive regulation of unknown nonlinear dynamical systems via dynamic neural networks. *IEEE Trans. Syst. Man Cybern.* **25**, 1578–1594 (1995)
28. Wang, C., Liu, H., Lin, T.: Direct adaptive fuzzyneural control with state observer and supervisory controller for unknown nonlinear dynamical systems. *IEEE Trans. Fuzzy Syst.* **10**, 39–49 (2002)
29. Leu, Y., Wang, W., Lee, T.: Observer-based direct adaptive fuzzy-neural control for nonaffine nonlinear systems. *IEEE Trans. Neural Netw.* **16**, 853–861 (2005)
30. Phan, P., Gale, T.J.: Direct adaptive fuzzy control with a self-structuring algorithm. *Fuzzy Sets and Systems* **159**, 871–899 (2008)
31. Christodoulou, M.A., Theodoridis, D.C., Boutalis, Y.S.: Building optimal fuzzy dynamical systems description based on recurrent neural network approximation. In: *Proc. Int. Conf. of Networked Distributed Systems for Intelligent Sensing and Control*. Kalamata, Greece (2007)
32. Boutalis, Y.S., Theodoridis, D.C., Christodoulou, M.A.: A new neuro FDS definition for indirect adaptive control of unknown nonlinear systems using a method of parameter hopping. *IEEE Trans. Neural Netw.* **20**(4), 609–625 (2009)
33. Theodoridis, D.C., Boutalis, Y.S., Christodoulou, M.A.: Indirect adaptive control of unknown multi variable nonlinear systems with parametric and dynamic uncertainties using a new neuro-fuzzy system description. *Int. J. Neural. Syst.* **20**(2), 129–148 (2010)
34. Cong, S., Liang, Y.: PID-like neural network nonlinear adaptive control for uncertain multivariable motion control system. *IEEE Trans. Ind. Electron.* **56**(10), 3872–3879 (2009)
35. Mbede, J.B., Huang, X., Wang, M.: Robust neural-fuzzy sensor based motion control among dynamic obstacles for robot manipulators. *IEEE Trans. Fuzzy Syst.* **11**, 249–260 (2003)
36. Patino, H.D., Carelli, R., Kuchen, B.R.: Neural networks for advanced control of robot manipulators. *IEEE Trans. Neural Netw.* **13**, 343–354 (2002)
37. Kelly, W., et al.: Neuro-fuzzy control of a robotic arm. In: *Proceedings of the Artificial Neural Networks in Engineering Conference*, pp. 837–842. St. Louis, MO, 10–13 Nov 1996
38. Beom, H.R., Cho, H.S.: A sensor-based navigation for a mobile robot using fuzzy-logic and reinforcement learning. *IEEE Trans. Syst. Man Cybern.* **25**(3), 464–477 (1995)
39. Lee, C.-H., Chiu, M.-H.: Recurrent neuro fuzzy control design for tracking of mobile robots via hybrid algorithm. *Expert. Syst. Appl.* **36**, 8993–8999 (2009)
40. Rusu, P., Petriu, E.M., Whalen, T.E., Crnell, A., Spoelder, H.J.W.: Behavior-based neuro-fuzzy controller for mobile robot navigation. *IEEE Trans. Instrum. Meas.* **52**(4), 1335–1340 (2003)
41. Martins, F.N., Celeste, W.C., Carelli, R., Sarcinelli-Finho, M., Bastos-Filho, T.F.: An adaptive dynamic controller for autonomous mobile robot trajectory tracking. *Control Eng. Pract.* **16**, 1354–1363 (2008)
42. Wang, J.S., Lee, C.S.G.: Self-adaptive recurrent neuro-fuzzy control of an autonomous underwater vehicle. *IEEE Trans. Rob. Autom.* **19**(2), 283–295 (2003)
43. Xu, B., Pandian, S.R., Sakagami, N., Petry, F.: Neuro-fuzzy control of underwater vehicle-manipulator systems. *J. Franklin Inst.* **349**, 1125–1138 (2012)
44. Suresh, S., Kannan, N., Sundararajan, N., Saratchandran, P.: Neural adaptive control for vibration suppression in composite fin-tip of aircraft. *Int. J. Neural. Syst.* **18**(3), 219–231 (2008)
45. Collani, Y.V., Zhang, J., Knoll, A.: A neuro-fuzzy solution for fine-motion control based on vision and force sensors. Technical Report, Department of Technology, University of Bielefeld (1997)
46. Lin, F.-J., Chou, P.-H., Shieh, P.-H., Chen, S.-Y.: Robust control of an LUSM-based $X - Y - \theta$ motion control stage using an adaptive interval type-2 fuzzy neural network. *IEEE Trans. Fuzzy Syst.* **17**(1), 24–38 (2009)
47. Wu, C.S., Gao, J.Q.: Vision-based neuro-fuzzy control of weld penetration in gas tungsten arc welding of thin sheets. *Int. J. Model. Ident. Control* **1**(2) (2006)

48. Lewis, J.P.: Fast normalized cross-correlation. In: Proceedings of Vision Interface, pp. 120–123 (1995)
49. Welch, G., Bishop, G.: An introduction to the Kalman Filter. In: Dept. Comp. Sci., Univ. North Carolina, Chapel Hill, TR95-041 (2000)
50. Ranganathan, J., Semke, W.: Three-axis gimbal surveillance algorithms for use in small UAS. In: Proceedings of the ASME International Mechanical Engineering Conference and Exposition, IMECE2008-67667 (2008)
51. Semke W., Ranganathan, J., Buisker, M.: Active gimbal control for surveillance using small unmanned aircraft systems. In: Proceedings of the International Model analysis Conference (IMAC) XXVI: A Conference and Exposition on Structural Dynamics (2008)